

Высокочастотный синусоидальный сигнал с частотой 100 Гц и белый шум.

Высокочастотный синус представляет собой периодическую помеху, аналогичную фону от электрических сетей. Белый шум – это случайные помехи, напоминающие шипение микрофона. Добавление этих шумов в сигнал позволяет проверить, как различные фильтры справляются с разными видами помех.

Рекурсивный фильтр (НЧ фильтр временной области)

Рекурсивный фильтр работает непосредственно с отсчётами сигнала во временной области, без преобразования его в частотный спектр. Это низкочастотный фильтр, который сглаживает сигнал, постепенно "забывая" быстрые изменения. Его работа основана на формуле:

$$y[i] = \alpha * x[i] + (1 - \alpha) * y[i-1]$$

Здесь $y[i]$ – текущее выходное значение, $x[i]$ – текущий входной сигнал, а $y[i-1]$ – предыдущее выходное значение. Параметр α (от 0 до 1) определяет степень сглаживания: чем он меньше, тем сильнее фильтр подавляет высокие частоты. Если $\alpha = 1$, фильтр не влияет на сигнал, а если α близко к 0, сигнал становится сильно сглаженным.

Чтобы превратить этот низкочастотный фильтр в высокочастотный, нужно вычесть сглаженный сигнал из исходного: $y_highpass[i] = x[i] - y_lowpass[i]$

Этот метод называется "рекурсивный высокочастотный фильтр" и позволяет выделить высокочастотные компоненты, убирая низкие.

Окно Блэкмана – одна из самых популярных оконных функций в цифровой обработке сигналов. Оно обеспечивает очень низкий уровень боковых лепестков (до - 58 дБ), что делает его идеальным для задач, где важно сильно подавить нежелательные частоты, например, при выделении слабых сигналов на фоне шума. Сглаживает резкие края коэффициентов фильтра. Давит боковые лепестки. Но размазывает основную зону фильтрации.

КИХ-фильтр – цифровой фильтр с конечной импульсной характеристикой, который обрабатывает сигнал путём свёртки с фиксированным набором коэффициентов, не используя обратную связь. КИХ-фильтр с окном Блэкмана (ВЧ фильтр частотной области) фильтр с конечной импульсной характеристикой – проектируется в частотной области. В отличие от рекурсивного, он не использует обратную связь, а работает через свёртку сигнала с заранее рассчитанными коэффициентами. Формула для сглаживания коэффициентов:

$$w[n] = 0.42 - 0.5 * \cos(2\pi n/(N-1)) + 0.08 * \cos(4\pi n/(N-1))$$

Она применяется к $h[i]$, чтобы уменьшить эффект Гиббса – $h_windowed[i] = h[i] * w[i]$. Коэффициенты:

0.42 – обеспечивает "базовый уровень" окна.

-0.5 – гасит основные колебания (первая гармоника).

+0.08 – корректирует вторую гармонику, дополнительно снижая лепестки.

Импульсная характеристика – это набор коэффициентов, определяющих, как фильтр реагирует на входной сигнал. Для высокочастотного фильтра коэффициенты рассчитываются по формуле:

$$h[i] = -\sin(2\pi \cdot f_c \cdot (i-M)/f_s) / (\pi \cdot (i-M)) \text{ для } i \neq M$$
$$h[M] = 1 - 2f_c/f_s \text{ для центрального элемента } (M = N // 2)$$

N – порядок фильтра (количество коэффициентов). Чем больше N , тем резче переход между полосами, но тем больше задержка сигнала.

f_c – частота среза (например, 50 Гц), разделяющая полосу пропускания и полосу подавления.

f_s – частота дискретизации (например, 1000 Гц).

Идеальный **фильтр** имеет бесконечную импульсную характеристику, но в цифровой обработке мы ограничены конечным числом коэффициентов. Простое обрезание идеальной характеристики вызывает эффект Гиббса – нежелательные выбросы в частотной характеристике фильтра. Окно Блэкмана умножается на коэффициенты фильтра, чтобы сгладить эти выбросы и уменьшить боковые лепестки в частотной характеристике.

Окна применяются при проектировании цифровых фильтров для уменьшения эффекта Гиббса – нежелательных выбросов и колебаний в частотной характеристике. Когда мы обрезаем идеальную бесконечную импульсную характеристику фильтра до конечного числа коэффициентов, резкое обрезание вызывает искажения в частотной области. Оконная функция плавно уменьшает амплитуду коэффициентов к краям, что снижает уровень боковых лепестков и делает переход между полосами более гладким.

Разные окна дают разный компромисс между:

Шириной главного лепестка (определяет резкость перехода между полосами).
Уровнем боковых лепестков (влияет на подавление нежелательных частот).

Чем сильнее окно подавляет боковые лепестки, тем шире становится главный лепесток, и наоборот. Выбор окна зависит от требований к фильтру: если нужно максимальное подавление помех, используют окна с низкими боковыми лепестками (например, Блэкмана), если важна резкость перехода – окна с узким главным лепестком (например, прямоугольное).

Сильное подавление боковых лепестков. Широкий главный лепесток (хуже разделяет близкие частоты, чем прямоугольное окно)

Применяется для подавления гула сети 50/60 Гц, в радиосвязи для подавления внеполосных помех. Выделения высокочастотных компонентов, таких как шипящие звуки в речи, для выделения полезных биосигналов (ЭКГ, ЭЭГ) на фоне шумов. Но из-за широкого главного лепестка может "размазывать" близкие спектральные компоненты.

Полосы – это ключевые частотные диапазоны, определяющие работу фильтра. Их три типа.

Полоса пропускания ($f > 50$ Гц) – частоты, которые фильтр пропускает почти без изменений (шум 100 Гц проходит).

Полоса подавления ($f < 50$ Гц) – частоты, которые фильтр ослабляет (сигнал 10 Гц подавляется).

Полоса перехода – область вокруг 50 Гц, где фильтр плавно меняет коэффициент передачи.

Чем уже полоса перехода, тем "резче" фильтр (требует больше коэффициентов).

Специальные фильтры – это нестандартные алгоритмы, предназначенные для узких задач. Например:

Медианный фильтр – удаляет резкие выбросы (импульсные помехи), заменяя каждый отсчет медианой из его окрестности.

Фильтр Калмана – используется в системах управления и навигации для оценки состояния системы с учётом шумов.

Зачем нужно сглаживание коэффициентов (оконная функция)?

Проблема:

При проектировании КИХ-фильтра мы обрезаем идеальную бесконечную импульсную характеристику до конечного числа коэффициентов (N). Это вызывает эффект Гиббса – нежелательные выбросы и колебания в частотной характеристике фильтра (высокие боковые лепестки).

Решение:

Умножение коэффициентов $h[i]$ на оконную функцию (например, Блэкмана) плавно уменьшает их амплитуду к краям. Это снижает уровень боковых лепестков (улучшает подавление нежелательных частот). Делает переход между полосами более плавным. Но расширяет главный лепесток (ухудшает резкость перехода).

Пример:

Без окна боковые лепестки могут быть -20 дБ, с окном Блэкмана – до -58 дБ.

Итого

В сигнале были два типа помех – высокочастотный синус (периодический) и белый шум (случайный), чтобы проверить работу фильтров на разных типах шумов. Рекурсивный фильтр – это низкочастотный фильтр временной области, который сглаживает сигнал, запоминая предыдущие значения. Параметр α регулирует степень сглаживания. Высокочастотная версия получается вычитанием низкочастотной составляющей из исходного сигнала. КИХ-фильтр с окном Блэкмана – это высокочастотный фильтр, работающий через свёртку с коэффициентами.

```

import numpy as np # Импорт библиотеки NumPy для численных операций
import matplotlib.pyplot as plt # Импорт библиотеки Matplotlib для визуализации
данных

# Параметры сигнала
Fs = 1000 # Частота дискретизации (Гц)
t = np.arange(0, 1, 1/Fs) # Временной вектор от 0 до 1 секунды с шагом 1/Fs
f_low = 10 # Частота низкочастотного полезного сигнала (Гц)
f_high = 100 # Частота высокочастотного шума (Гц)

# Генерация сигнала
signal = np.sin(2 * np.pi * f_low * t) # Генерация синусоидального полезного
сигнала

# Визуализация исходного сигнала
plt.figure(figsize=(10, 4)) # Создание нового окна для графика размером 10x4
дюймов
plt.plot(t, signal) # Построение графика сигнала в зависимости от времени
plt.title("Исходный сигнал") # Установка заголовка графика
plt.xlabel("Время (с)") # Установка подписи оси X
plt.ylabel("Амплитуда") # Установка подписи оси Y
plt.grid() # Включение сетки на графике
plt.show() # Отображение графика

# Добавление шума к сигналу
signal += 0.5 * np.sin(2 * np.pi * f_high * t) # Добавление высокочастотной
шумовой компоненты
signal += 0.2 * np.random.normal(size=t.shape) # Добавление аддитивного белого
гауссовского шума

# Визуализация сигнала с шумом
plt.figure(figsize=(10, 4)) # Создание нового окна для графика
plt.plot(t, signal) # Построение графика зашумленного сигнала
plt.title("Исходный сигнал с шумом") # Установка заголовка графика
plt.xlabel("Время (с)") # Установка подписи оси X
plt.ylabel("Амплитуда") # Установка подписи оси Y
plt.grid() # Включение сетки
plt.show() # Отображение графика

def blackman_window(N):
    """Генерация окна Блэкмана."""
    n = np.arange(N) # Создание массива индексов от 0 до N-1
    # Расчет значений окна Блэкмана по формуле
    return 0.42 - 0.5 * np.cos(2 * np.pi * n / (N - 1)) + 0.08 * np.cos(4 * np.pi
* n / (N - 1))

def highpass_blackman(N, fc, fs):
    """Проектирование ВЧ фильтра с окном Блэкмана."""
    M = N // 2 # Определение центрального индекса (половина порядка фильтра)
    h = np.zeros(N) # Инициализация массива для импульсной характеристики нулями
    # Расчет идеальной импульсной характеристики НЧ фильтра

```

```

for i in range(N):
    if i == M: # Особый случай для центрального элемента (избегание деления
на ноль)
        h[i] = 1 - 2 * fc / fs
    else:
        h[i] = -np.sin(2 * np.pi * fc * (i - M) / fs) / (np.pi * (i - M))
# Умножение на окно Блэкмана для сглаживания
window = blackman_window(N) # Генерация окна Блэкмана
h *= window # Применение окна к импульсной характеристике
# Инвертируем для получения ВЧ фильтра (спектральная инверсия)
h = -h
h[M] += 1 # Коррекция центрального элемента для ВЧ
# Нормировка (необязательно, но часто используется для сохранения амплитуды)
# h /= np.sum(np.abs(h)) # Раскомментируйте, если нужна нормировка
return h # Возвращение импульсной характеристики ВЧ фильтра

# Параметры фильтра
N = 101 # Порядок фильтра (нечетный для симметрии типа I)
fc = 50 # Частота среза (Гц)
fs = Fs # Частота дискретизации (используем ту же, что и для сигнала)

# Создание ВЧ фильтра
h_hp = highpass_blackman(N, fc, fs) # Вызов функции для проектирования фильтра

# Применение ВЧ фильтра (свертка)
# mode='same' возвращает результат свертки той же длины, что и исходный сигнал
filtered_hp = np.convolve(signal, h_hp, mode='same')

# Визуализация результата ВЧ фильтрации
plt.figure(figsize=(10, 4)) # Создание нового окна для графика
plt.plot(t, signal, label="Исходный сигнал") # Построение исходного сигнала
plt.plot(t, filtered_hp, label="После ВЧ фильтра") # Построение отфильтрованного
сигнала
plt.title("Результат ВЧ фильтрации (окно Блэкмана)") # Установка заголовка
plt.xlabel("Время (с)") # Подпись оси X
plt.ylabel("Амплитуда") # Подпись оси Y
plt.legend() # Отображение легенды
plt.grid() # Включение сетки
plt.show() # Отображение графика

# Расчет и визуализация АЧХ ВЧ фильтра
N = np.fft.fft(h_hp, 2048) # Вычисление ДПФ импульсной характеристики (с
дополнением нулями до 2048 точек для лучшего разрешения)
freq = np.fft.fftfreq(2048, 1/fs)[:1024] # Расчет частотной оси (берем только
положительные частоты)
plt.figure(figsize=(10, 4)) # Создание нового окна для графика
plt.plot(freq, 20 * np.log10(np.abs(N[:1024]))) # Построение АЧХ в децибелах
plt.axvline(fc, color='r', linestyle='--', label=f'Fc = {fc} Гц') # Вертикальная
линия на частоте среза
plt.title("АЧХ ВЧ фильтра (окно Блэкмана)") # Установка заголовка
plt.xlabel("Частота (Гц)") # Подпись оси X

```

```

plt.ylabel("Усиление (дБ)") # Подпись оси Y
plt.ylim(-60, 5) # Установка пределов оси Y для лучшей видимости
plt.grid() # Включение сетки
plt.legend() # Отображение легенды
plt.show() # Отображение графика

def recursive_lowpass(x, alpha):
    """Однородный рекурсивный НЧ фильтр первого порядка."""
    y = np.zeros_like(x) # Инициализация выходного массива нулями той же формы,
    что и входной
    y[0] = x[0] # Начальное условие: первый элемент выхода равен первому
    элементу входа
    # Рекурсивный расчет выходных значений
    for i in range(1, len(x)):
        y[i] = alpha * x[i] + (1 - alpha) * y[i-1] # Формула рекурсивного
    фильтра
    return y # Возвращение отфильтрованного сигнала

# Параметры рекурсивного фильтра
alpha = 0.1 # Коэффициент сглаживания (0 < alpha < 1). Меньше alpha - сильнее
сглаживание.

# Применение рекурсивного НЧ фильтра
filtered_recursive = recursive_lowpass(signal, alpha) # Вызов функции фильтрации

# Визуализация результата рекурсивной фильтрации
plt.figure(figsize=(10, 4)) # Создание нового окна для графика
plt.plot(t, signal, label="Исходный сигнал") # Построение исходного сигнала
plt.plot(t, filtered_recursive, label="После рекурсивного фильтра") # Построение
отфильтрованного сигнала
plt.title("Результат рекурсивной фильтрации") # Установка заголовка
plt.xlabel("Время (с)") # Подпись оси X
plt.ylabel("Амплитуда") # Подпись оси Y
plt.legend() # Отображение легенды
plt.grid() # Включение сетки
plt.show() # Отображение графика

# Расчет и визуализация АЧХ рекурсивного НЧ фильтра
w = np.linspace(0, np.pi, 1000) # Создание вектора нормированных частот от 0 до
pi
# Расчет теоретической передаточной функции H(z) на единичной окружности z =
exp(j*w)
H = alpha / (1 - (1 - alpha) * np.exp(-1j * w))
freq = w * Fs / (2 * np.pi) # Преобразование нормированных частот в Гц
plt.figure(figsize=(10, 4)) # Создание нового окна для графика
plt.plot(freq, 20 * np.log10(np.abs(H))) # Построение АЧХ в децибелах
plt.title("АЧХ рекурсивного НЧ фильтра") # Установка заголовка
plt.xlabel("Частота (Гц)") # Подпись оси X
plt.ylabel("Усиление (дБ)") # Подпись оси Y
plt.grid() # Включение сетки
plt.show() # Отображение графика

```